2019 IEEE International Conference on Innovative Computing (ICIC)

A Novel Time and Space Complexity Efficient Variant of Counting-Sort Algorithm

Asad R. Usmani
Department of Computer Science
University of Management and Technology
Lahore, Pakistan
asad.usmani@umt.edu.pk

***Abstract*—Sorting is a well-known problem, which is most commonly discussed in Algorithms. Although, there exist many algorithms to solve the sorting problem but Counting-sort algorithm has its own importance due to its linear time-complexity, which is $O(n+k)$ with $2n+k$ space-complexity. Moreover, depending upon the feasibility of input data, It is the most efficient sorting algorithm available in terms of time if $n > k$ holds but a significant value of $k-n$ restricts its usage in practice due to its high memory and computational demands in case of violation. So, in this paper, we have proposed a novel variant of trivial Counting-sort algorithm, which is comparatively not only time but space efficient as well. The time and space complexity of our proposed ARU-Counting-sort algorithm is $O(n+\sqrt{k})$ and $2n+2\sqrt{k}$ respectively.**

***Index Terms*—Counting Sort, Sorting Algorithms, Indexing, Complexity Analysis, Optimization**

I. Introduction

Arrangement in an ascending or descending order of a list of objects is called sorting. Sorting is one of the well-known problems which is commonly discussed not only in Algorithms but people also deal with it in their daily life. More importantly, a lot of applications from various fields of computing [1]–[3] like Databases face sorting problem and demands sorting operation in order to fulfill user's adequate requirements. For example, now a days, every user needs a quicker response while interacting with an information system. Searching is a very common operation which is being done behind almost 80% queries posted by a user [4]. Consequently, that information system have to maintain its data in an ordered fashion for responding its users queries more quickly. Thus, sorting becomes a necessity to have a fast and efficient search of any data-processing system, where quick response is highly desirable. Due to this reason, people have always tried to improve different sorting algorithm [5]–[9].

In this regard, over the decades, a lot of remarkable work has already been done and still carried on by many computer scientists and mathematicians on sorting algorithms and their applications [10]–[13]. Bubble-sort, Insertion-sort, Selection-sort, Merge-sort, Quick-sort, Bitonic-sort, Random-sort and Counting-sort are some most commonly used sorting algorithms [14], [15]. Among those, Quick-sort is being considered one of the fastest sorting algorithm available near most of the people [17]. Surprisingly, a comprehensive theoretical and computational analysis shows that our newly novel proposed sorting algorithm named as ARU-Counting-sort (ARU are author's name initials) is not only better than trivial Counting-sort but also other best sorting algorithms (including Quick-sort) in terms of time-complexity. Sections II and III will describe the Counting-sort and ARU-Counting-sort respectively.

✪ Funded by University of Management and Technology, Lahore, Pakistan.

II. Counting-Sort Algorithm

Counting sort [16] is a stable, non in-place and linear time sorting algorithm, which is used to sort a finite number of integer-type elements. Counting-sort algorithm is a basic building block of many data processing applications because of its fast speed and high efficiency [28]. Here, we have a sequence of steps for trivial Counting-sort algorithm to know how exactly it works.

Algorithm 1 Counting-Sort Algorithm

```
procedure COUNTINGSORT(A, B, C, k)                ▷
    There A, B and C are the arrays of length n, n and k+1
    respectively, where k is the maximum element in array A,
    which is needed to be sorted.
    i := 0;   j := 0;
    while i ≤ k do
        C[i] := 0;
        i := i + 1;
    while j < n do
        C[A[j]] := C[A[j]] + 1;
        j := j + 1;
    i := 1;   j := n − 1;
    while i ≤ k do
        C[i] := C[i] + C[i − 1];
        i := i + 1;
    while j ≥ 0 do
        B[C[A[j]]] := A[j];
        C[A[j]] := C[A[j]] − 1;
        j := j − 1;
    return B
```

978-1-7281-4682-9/19/$31.00 © 2019 IEEE

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.

This algorithm assumes that all the n input elements are non-negative and integer type, which are required to be sorted. Those elements lie between a range from 0 to k, where k is the maximum element of all the input integers. The basic idea of counting sort is that it just tries to relocate every element in the input array depending upon its integer value and then finds its correct position to get a sorted output array in the end. The correct position of any element is determined by counting the total number of elements larger and smaller than that element. Here, we need a complete study of its complexity analysis in terms of both time and space for its comparison with other sorting algorithms.

A. Time Complexity Analysis

1) An auxiliary array C of size k is being initialized with zero in the loop of $lines\ 2-5$ and it takes $O(k)$ time.
2) For each element in the input array A, says x, an occurrence of it is being counted in auxiliary array C in the loop of $lines\ 6-8$ and it takes $O(n)$ time.
3) A cumulative operation on auxiliary array C of size k is being performed in the loop of $lines\ 9-12$ and its time-complexity is $O(k)$.
4) For each element in the input array A, says x, its exact index is being found and then relocated into the sorted output array B in the loop of $lines\ 13-17$ and it takes $O(n)$ time.
5) Total time complexity = $O(k)+O(n)+O(k)+O(n) = O(n+k)$

B. Space Complexity Analysis

1) We need n memory elements for input array A.
2) We require k memory elements for auxiliary array C.
3) We need n memory elements for output array B.
4) Total space complexity = $(n+k+n) = 2n+k$

So, complexity of this algorithm is entirely dependent on two variables n and k. Where n is representing total count of input elements and k denotes the maximum integer value among those elements. If we have an input array of n integers such that $k \leq n$ then both time and space complexity of this algorithm will be $O(n)$ as $k = O(n)$, which is the best case of this algorithm. The best case of this algorithm proves that it can perform sorting in linear time $O(n)$. The major drawback of this algorithm is another case, suppose we have n integers such that $n^2 \leq k$ then both time and space complexity of this algorithm will be $O(n^2)$ as $k = O(n^2)$. Unfortunately, this complexity analysis makes this algorithm even worse than all other sorting algorithms. As Quick sort, Heap sort and Merge sort have $O(n \log n)$ time and Insertion sort, Bubble sort as well as Selection sort have same $O(n^2)$ time but have n space-complexity instead. We take an example to see a more clear view.

$$Let\ A = \{17, 45, 2, 64, 11, 19, 43, 9, 100, 78\}$$
$$where\ n = |A| = 10\ and\ k = max(A) = 100$$

When we apply Counting-sort algorithm to sort above input array A, we find both time and space complexities are $O(100)$. On the other hand, if use Insertion-sort or Bubble-sort on same input array A, then those algorithms run in same $O(100)$ time but take 10 space-complexity only. Clearly, Counting-sort algorithm is consuming n^2 space while others are taking only linear space-complexity n. More precisely, It may become even worse in this scenario for $n = 10$ and $k = 1000$. By keeping in mind this drawback, we have proposed a new variant (ARU-Counting-sort) of Counting-sort algorithm. It is much better to use than trivial Counting-sort algorithm in complexity analysis. Its details are discussed immediately in next section III.

III. Proposed ARU-Counting-Sort Algorithm

Algorithm 2 ARU-Counting-Sort Algorithm

1: **procedure** ARUCountingSort(A, B, Q, R, k) ▷ There A, B, Q and R are the arrays of length n, n, $m+1$ and $m+1$ respectively, where $m = \lceil\sqrt{k}\rceil$ and k is the maximum element in array A, which is needed to be sorted.
2: $i := 0;\quad j := 0;\quad m := \lceil\sqrt{k}\rceil;$
3: **while** $i <= m$ **do**
4: $Q[i] := 0;$ ▷ Q is representing quotient array
5: $R[i] := 0;$ ▷ R is representing remainder array
6: $i := i + 1;$
7: **while** $j < n$ **do**
8: $Q[A[j]/m] := Q[A[j]/m] + 1;$
9: $R[A[j]\%m] := R[A[j]\%m] + 1;$
10: $j := j + 1;$
11: $i := 1;$
12: **while** $i <= m$ **do**
13: $Q[i] := Q[i] + Q[i-1];$
14: $R[i] := R[i] + R[i-1]];$
15: $i := i + 1;$
16: $j := n - 1;$
17: **while** $j >= 0$ **do**
18: $d := A[j]\%m;$
19: $R[d] := R[d] - 1;$
20: $B[R[d]] := A[j];$ ▷ B is an auxiliary array
21: $j := j - 1;$
22: $i := n - 1;$
23: **while** $i >= 0$ **do**
24: $d := B[i]/m;$
25: $Q[d] := Q[d] - 1;$
26: $A[Q[d]] := B[i];$
27: $i := i - 1;$
28: **return** A

A. Proof of Concept

1) Let $A = \{a_0, a_1, a_2, \ldots, a_{n-1}\}$ is an array containing all unsorted list of n elements, which need to be sorted.
2) Now, we will divide those n elements present in A array into m chunks, where k is the maximum element in A array and $m = \lceil\sqrt{k}\rceil$, such that:
$each\ element\ a_i \in B_j\ chunk\ if\ \lceil\frac{a_i}{m}\rceil \equiv j;\ where\ 0 \leq i < n\ and\ 0 \leq j \leq m.$

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.

$$\overbrace{\underbrace{a_3, a_9, a_5, a_1}_{B_0} \quad \underbrace{a_6, a_8}_{B_1} \quad \underbrace{a_2}_{B_2} \quad \cdots \quad \underbrace{a_0, a_7, a_4}_{B_m}}^{\text{All } m+1 \text{ chunks of array } A}$$

Obviously, elements having same quotient when divided by m will belong to the same chunk. Probably, any chunk B_j may be empty depending upon the input data. Most importantly, it can easily be ensured that every element $(a_i \in B_j) < (a_i \in B_{j+1}) \cdots < (a_i \in B_m)$ in case of ascending order. For example, all elements a_3, a_9, a_5 and a_1 in chunk B_0 are smaller than any element in B_1, B_2 and higher chunks. That means, all $m+1$ chunks are in an ordered fashion till now.

3) Afterwards, we will be only required to sort the elements within each chunk B_j. So that, all the array will become in sorted order. In this regard, we shuffle all the elements in array A according to their remainders. The element with greater remainder will occupy higher index or position in auxiliary output array as compared to element having smaller remainder when both are divided by m. We actually rearrange all the elements of input array A using their remainders in $lines\#16-21$ to ensure this. For any two elements a_s and a_r belong to the same chuck B_j such that $s < r$ if $(a_s\%m) \leq (a_r\%m)$ where $0 \leq s < r < n$. In this way, all elements belong to same chunk will be arranged in an ordered fashion for every chunk and ultimately, the final array list will become sorted.
4) Cautiously, We have performed step 3 $(line\#16-21)$ before step 2 $(line\#22-27)$ in our algorithm 2. In fact, we described their order is such a way for better understanding the real thought behind this novel Counting-sort algorithm here.

B. Time Complexity Analysis

1) Both auxiliary arrays Q and R of each size $\sqrt{k}$ are being initialized with zero in the loop of $lines$ $3-6$ and it takes $O(\sqrt{k})$ time.
2) For each element in the input array A, says x, an occurrence of its quotient and remainder are being counted in both auxiliary arrays Q and R respectively in the loop of $lines$ $7-10$ and it takes $O(n)$ time.
3) A cumulative operation on both auxiliary arrays Q and R of each size $\sqrt{k}$ are being computed in the loop of $lines$ $11-15$ and it takes $O(\sqrt{k})$ time.
4) Now, for each element in the input array A, says x, it is being shifted into a temporary array B based on its remainder value in the loop of $lines$ $16-21$ and time-complexity of this step is $O(n)$.
5) Finally, for each element in the input array A, says x, its exact index is being found using B and then relocated into the sorted output array B in the loop of $lines$ $22-27$ and it also takes $O(n)$ time.
6) Total time complexity = $O(\sqrt{k}) + O(n) + O(\sqrt{k}) + O(n) + O(n) = O(n + \sqrt{k})$

C. Space Complexity Analysis

1) We require n memory elements for both input arrays A and B separately.
2) We need $\sqrt{k}$ memory elements for each quotient Q and remainder R array.
3) Total space complexity = $(n+n+\sqrt{k}+\sqrt{k}) = 2n+2\sqrt{k}$

D. An Example

$Let\ A = \{17, 2, 38, 63, 19, 44, 29, 11\}$

$where\ n = |A| = 8, k = max(A) = 63\ and\ \lceil\sqrt{k}\rceil = 8$

1) After initialization of both quotient and remainder arrays in the loop of $lines$ $3-6$, the state of all arrays are given below for indexes $0-7$.

Arrays	0	1	2	3	4	5	6	7
A	17	2	38	63	19	44	29	11
B	0	0	0	0	0	0	0	0
Q	0	0	0	0	0	0	0	0
R	0	0	0	0	0	0	0	0

2) Now, for every element is array A, says x, we will find its quotient as well as remainder and then update Q and R arrays respectively in the loop of $lines$ $7-10$.

Arrays	0	1	2	3	4	5	6	7
A	17	2	38	63	19	44	29	11
B	0	0	0	0	0	0	0	0
Q	1	1	2	1	1	1	0	1
R	0	1	1	2	1	1	1	1

3) In the loop of $lines$ $11-15$, the cumulative operation is performed on both quotient Q and remainder R arrays and states of these arrays are updated then.

Arrays	0	1	2	3	4	5	6	7
A	17	2	38	63	19	44	29	11
B	0	0	0	0	0	0	0	0
Q	1	2	4	5	6	7	7	8
R	0	1	2	4	5	6	7	8

4) Here, we are shifting the elements based on their remainders from array A to temporary array B in the loop of $lines$ $16-21$.

Arrays	0	1	2	3	4	5	6	7
A	17	2	38	63	19	44	29	11
B	17	2	19	11	44	29	38	63
Q	1	2	4	5	6	7	7	8
R	0	0	1	2	4	5	6	7

5) Finally, we are again rearranging the elements based on their quotients from temporary array B to final desired sorted array A in the loop of $lines$ $22-27$.

Arrays	0	1	2	3	4	5	6	7
A	2	11	17	19	29	38	44	63
B	17	2	19	11	44	29	38	63
Q	0	1	2	4	5	6	7	7
R	0	0	1	2	4	5	6	7

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.

IV. Results and Discussions

Here, we can easily extract that our proposed novel efficient ARU-Counting-sort algorithm is much better than trivial Counting-sort algorithm in terms of both time and space complexities. Our algorithm have $O(n+\sqrt{k})$ time and $(2n+2\sqrt{k})$ space complexity as compared to inherited Counting-sort algorithm, which have $O(n+k)$ time and $(2n+k)$ space complexity. Suppose $n = 100$ and $k = 10000$, in this case the time and space complexity of trivial Counting-Sort algorithm is $O(100+10000)$ and $(200+10000)$ respectively. In contrast, our proposed algorithm will have $O(100+100)$ time and $(200+200)$ space complexity respectively. Clearly, not only time-complexity but also its space-complexity has been reduced from depending factor k (which exists in traditional Counting-sort algorithm) to $\sqrt{k}$. Which means that our algorithm will require less than four times of n memory elements unless until $\sqrt{k} \leq n$. Moreover, our algorithm ensures that its time-complexity is linear in terms of n and not dependent further on k factor till $\sqrt{k} \leq n$, which is now an advantage not only on trivial Counting-sort algorithm but also all over other available sorting algorithms and encourages people to use this algorithm in practice without any hesitation. Let's discuss the case when $n < \sqrt{k}$ then time and space complexity both will not remain depend on factor n anymore but on $\sqrt{k}$ instead. For example, if $n = 1000$ and $k = 10000000$ then the condition $n < \sqrt{k}$ will be true. Will in this case our proposed algorithm will act poorly as compared to other best sorting algorithms available?

As we know that the Quick, Merge and Heap sort, all have $\Theta(n \log n)$ time complexity as compared to Bubble, Selection and Insertion sort algorithms, which have $\Theta(n^2)$ [15]. Hence, we just need to compare our proposed algorithm with only best available sorting algorithms to answer the above question.

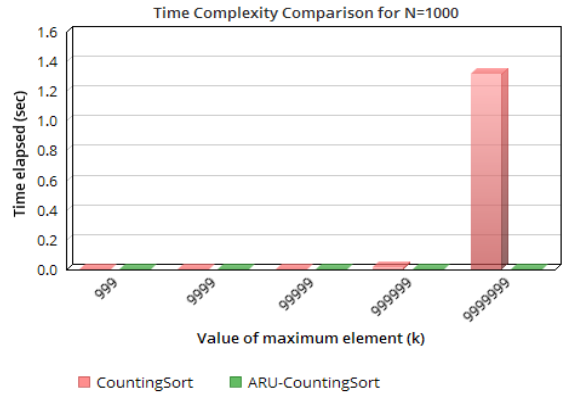

Figure 2. Shows the time elapsed for both algorithms for different values of k and $n = 1000$.

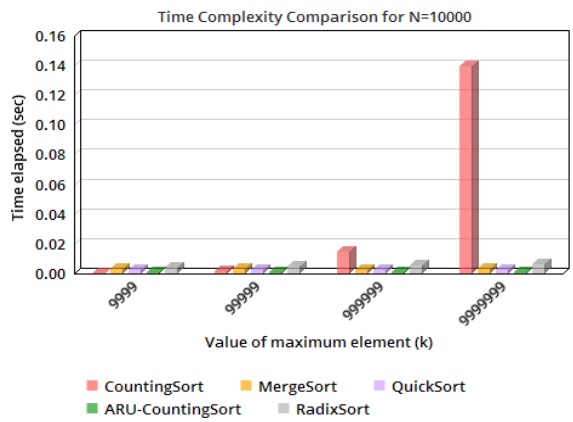

Figure 3. Shows the time elapsed by different algorithms including Counting-sort for different values of k and $n = 10000$.

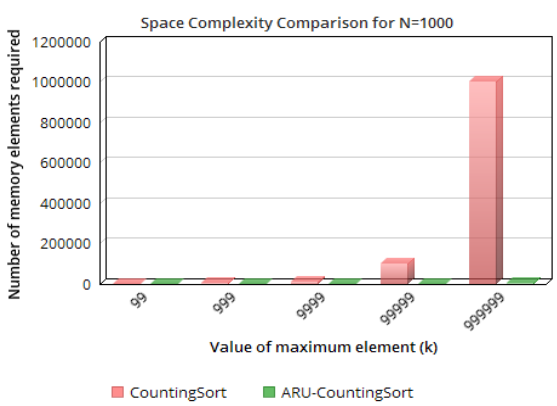

Figure 1. Shows the memory elements required to both algorithms for different values of k and $n = 1000$.

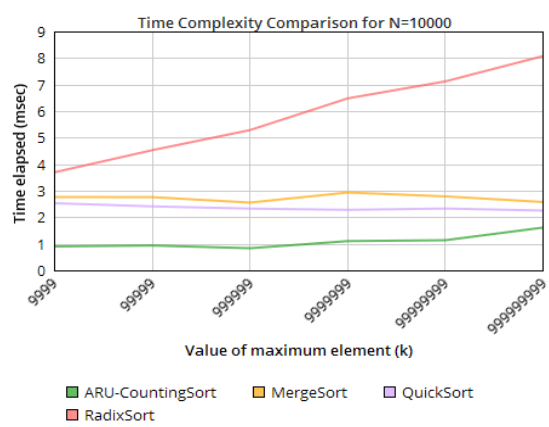

Figure 4. Shows the time elapsed by different algorithms excluding Counting-sort for different values of k and $n = 10000$.

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.

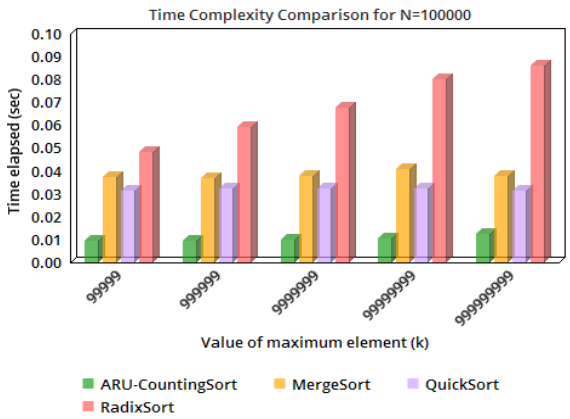

Figure 5. Shows the time elapsed by different algorithms for different values of k and $n = 100000$.

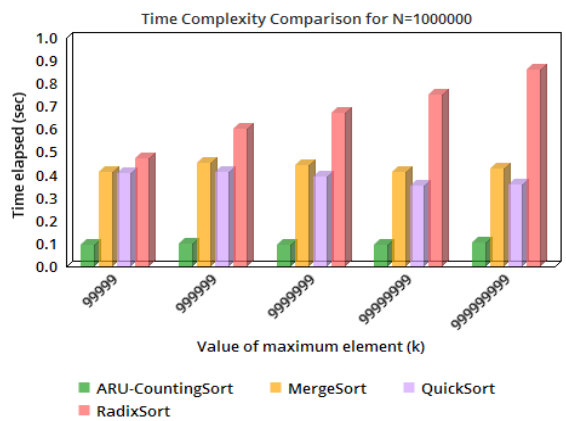

Figure 6. Shows the time elapsed by different algorithms for different values of k and $n = 1000000$.

We have taken the C/C++ implementations of all algorithms from here [18]. We can verify the desired results shown in above figures after running their codes on a desktop machine with specifications of Processor Core-i7, Memory 8GB with installed OS Windows 10 and IDE NetBeans 8.1.

Figure 1 shows a comparison of space-complexity of both trivial Counting-sort and new ARU-Counting-sort algorithms for a fixed value of $n = 1000$ and different values of k ranging from 99 to 999999. Clearly, as the value of k increases we can notice a lot of differences between both algorithm's space requirements. In case of $k = 999999$, our ARU-Counting-Sort will consume almost 250 times lesser memory elements than trivial Counting-sort.

On the other hand, figure 2 shows a comparison of time-complexity of both trivial Counting-sort and new ARU-Counting-sort algorithms for a fixed value of $n = 1000$ and different values of k ranging from 999 to 99999999. Significant differences between both algorithm's time-elapsed can be observed. Considering the value of $k = 9999999$, we can measure a speedup of more than 5000 times of our ARU-Counting-sort over inherited Counting-sort algorithm. The speedup can be calculated using below formula:

$$\text{Speedup} = (n + k)/(n + \sqrt{k})$$

It is a now a clear fact that trivial Counting-sort is worst than all other sorting algorithm as shown in figure 3. we can see the contrast of time-complexity of different sorting algorithms for a fixed value of $n = 10000$ and different values of k ranging from 9999 to 999999999.

For better interpretation, we compare the time-complexity of our ARU-Counting-sort with only other best sorting algorithms for a fixed value of $n = 10000$ and different values of k till $k = 999999999$. Evidently, ARU-Counting-sort have lesser time-elapsed to all other competitor algorithms. It is almost twice quicker than Quick-sort and Merge-sort, which can easily be extracted from the line graph mentioned in figure 4.

Both figures 4 and 5 are depicting the comparisons of time-complexity of ARU-Counting-sort and other algorithms for fixed values of $n = 100000$, $n = 100000$ and different values of k ranging from 9999 to 999999999 respectively. Even for the largest value of $k = 999999999$, ARU-Counting-sort is still better than best Quick-sort algorithm.

V. Conclusion and Future Work

For a couple of decades, people have implemented improved versions of Counting-sort algorithm using different environments like distributed computing [19], GPU computing [20] and hybrid approaches [21]. Instead of using those specific computing environments, its really need to have a more efficient sorting algorithm for a single CPU based environment. Thus, we have implemented an efficient novel variant of Counting-sort algorithm.

In previous sections, we have concluded that our newly introduced novel ARU-Counting-Sort is much better than its trivial Counting-sort algorithm in both time and space-complexity. ARU-Counting-sort has $O(n + \sqrt{k})$ and $O(2n + 2\sqrt{k})$ time and space complexity respectively instead of $O(n + k)$ and $O(2n + k)$, which the traditional Counting-sort have. Moreover, we have also analyzed that ARU-Counting-sort in also better than all existing best sorting algorithms like Quick-sort, Merge-sort and Radix-sort in case of time-complexity. One major drawback of ARU-Counting-sort over other sorting algorithms is that it still demands extra memory as compared to some of best in-place sorting algorithms. In case of $n == \sqrt{k}$, it requires exactly 4 times extra memory as other in-place sorting algorithms consume like Quick-sort. In both other cases, for $n > \sqrt{k}$ and $n < \sqrt{k}$ it needs less and more than 4 times extra memory in contrast to in-place Quick-sort algorithm respectively.

In general comparison, we know that the maximum value of 32-bits positive signed integer is less than 10^{10} and maximum value of 64-bits positive signed integer is less than 10^{19} [23]. So, maximum value of $\sqrt{k}$ in both above cases will be less than 10^5 and 10^{10} respectively. Which means that on 32-bits machine, more than 10 thousands elements will be sorted

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.

in $O(n)$ with 100% guarantee in any case using our ARU-Counting-sort algorithm because of $\sqrt{k} \leq n$. Consequently, time complexity of our ARU-Counting-sort algorithm will not be dependent of $\sqrt{k}$ variable anymore when the condition $\sqrt{k} \leq n$ will be true.

In conclusion, we can say that our ARU-Counting-sort algorithm is now best to use for sorting problems in $O(n)$ when $\sqrt{k} \leq n$ and still working better than other algorithms having $\Theta(n \log n)$ time-complexity in other case which may not occur frequently. For instant, if $n = 1000$ then till $k \leq 1000000$ our algorithm guarantee that the time-complexity will be $O(n)$. Which is very surprising as it has been proved earlier that sorting can never be done in $O(n)$ [22] and in other case it is still performing better than those algorithms having $\Theta(n \log n)$ time-complexity. Hence, we claim that our proposed ARU-Counting-sort is now the worldś best sorting algorithm in terms of efficiency. So, it can be used for any set of data especially for large set in order to get data sorted in $O(n)$ time-complexity without any hesitation where memory is not a much concern. Even though, we can also reduce 25% memory required by ARU-Counting-sort algorithm by performing quotient and remainder operations sequentially one after another using only one shared array of size $\sqrt{k}$. Overall, a lot of applications in every domain of computing, e.g, [24]–[27] which are based on sorting algorithms may achieve more efficiency using our proposed algorithm. So, in future, we have aim to definitely work on those applications and implement this algorithm on different computing environments as well.

Acknowledgment

I remained a bright student of Dr. Tahir Ejaz at PUCIT, University of the Punjab, Lahore, Pakistan during 2010-2012. Where, he taught me the course of "Advance Algorithms" during my M.Phil studies. I really enjoyed not only the course but also get involved into some research ideas in the area of Algorithms. From where, I am able to drive this fascinating novel idea of ARU-Counting-Sort algorithm. So, I really want to acknowledge Dr. Tahir Ejaz for polishing me to do this accomplished piece of work. Now, he is working as an Assistant Professor in University of Management and Technology, Lahore, Pakistan.

References

[1] Miao, Meixia, Jianfeng Wang, Sheng Wen, and Jianfeng Ma. "Publicly verifiable database scheme with efficient keyword search." Information Sciences 475 (2019): 18-28.

[2] Aronovich, Lior, Ron Asher, Eitan Bachmat, Haim Bitner, Michael Hirsch, and Shmuel T. Klein. "Systems and methods for efficient data searching, storage and reduction." U.S. Patent Application 10/282,257, filed May 7, 2019.

[3] Chen, He, Xiuxia Tian, and Cheqing Jin. "Fuzzy Searching Encryption with Complex Wild-Cards Queries on Encrypted Database." In Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data, pp. 3-18. Springer, Cham, 2018.

[4] Ramirez-Chacon, Miguel, Hugo Hidalgo-Silva, and Edgar Chavez. "Indexing and searching point clouds." Journal of Intelligent and Fuzzy Systems 34, no. 5 (2018): 3349-3358.

[5] S. White, N. Verosky and T. Newhall, "A CUDA-MPI Hybrid Bitonic Sorting Algorithm for GPU Clusters," 2012 41st International Conference on Parallel Processing Workshops, Pittsburgh, PA, 2012, pp. 588-589.

[6] Z. Yildiz, M. Aydin and G. Yilmaz, "Parallelization of bitonic sort and radix sort algorithms on many core GPUs," 2013 International Conference on Electronics, Computer and Computation (ICECCO), Ankara, 2013, pp. 326-329.

[7] S. Abdel-Hafeez and A. Gordon-Ross, "An Efficient O(N) Comparison-Free Sorting Algorithm," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 25, no. 6, pp. 1930-1942, June 2017.

[8] A. Uyar, "Parallel merge sort with double merging," 2014 IEEE 8th International Conference on Application of Information and Communication Technologies (AICT), Astana, 2014, pp. 1-5.

[9] K. J. Kim, S. J. Cho and J. Jeon, "Parallel quick sort algorithms analysis using OpenMP 3.0 in embedded system," 2011 11th International Conference on Control, Automation and Systems, Gyeonggi-do, 2011, pp. 757-761.

[10] M. Harris, S. Sengupta, and J. D. Owens. Parallel Prefix Sum (Scan) with CUDA. In H. Nguyen, editor, GPU Gems 3. Addison Wesley, 2007.

[11] Cederman, Daniel, and Philippas Tsigas. "A practical quicksort algorithm for graphics processors." In European Symposium on Algorithms, pp. 246-258. Springer, Berlin, Heidelberg, 2008.

[12] Sachnev, Vasiliy, Hyoung Joong Kim, Jeho Nam, Sundaram Suresh, and Yun Qing Shi. "Reversible watermarking algorithm using sorting and prediction." IEEE Transactions on Circuits and Systems for Video Technology 19, no. 7 (2009): 989-999.

[13] Coello, Carlos A. Coello, and Gary B. Lamont. Applications of multi-objective evolutionary algorithms. Vol. 1. World Scientific, 2004.

[14] Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms. MIT press, 2009.

[15] Aho, Alfred V., and John E. Hopcroft. The design and analysis of computer algorithms. Pearson Education, 1974.

[16] Wikipedia article: Counting sort. http://en.wikipedia.org/wiki/ Counting_sort.

[17] Cederman, Daniel, and Philippas Tsigas. "Gpu-quicksort: A practical quicksort algorithm for graphics processors." Journal of Experimental Algorithmics (JEA) 14 (2009): 4.

[18] Geeksforgeeks article: Sorting Algorithms Code. https://www.geeksforgeeks.org/sorting-algorithms/.

[19] W. Sun and Z. Ma, "Count Sort for GPU Computing," 2009 15th International Conference on Parallel and Distributed Systems, Shenzhen, 2009, pp. 919-924.

[20] Faujdar, Neetu, and Satya Prakash Ghrera. "Performance evaluation of parallel count sort using GPU computing with CUDA." Indian Journal of Science and Technology 9 (2016): 15.

[21] T. Umeda and S. Oya, "Performance Comparison of Open-Source Parallel Sorting with OpenMP," 2015 Third International Symposium on Computing and Networking (CANDAR), Sapporo, 2015, pp. 334-340.

[22] Han, Yijie. "Deterministic sorting in O (n log log n) time and linear space." In Proceedings of the thiry-fourth annual ACM symposium on Theory of computing, pp. 602-608. ACM, 2002.

[23] Wikipedia article: Integers. https://en.wikipedia.org/wiki/ 2,147,483,647.

[24] Y. Li, F. Sha, S. Wang and T. Hu, "The improvement of page sorting algorithm for music users in Nutch," 2016 IEEE/ACIS 15th International Conference on Computer and Information Science (ICIS), Okayama, 2016, pp. 1-4.

[25] Yao, Yuan. "Virtual Memory Streaming and Sorting in MapReduce Applications." PhD diss., 2018.

[26] Bowers, K. J. "Accelerating a particle-in-cell simulation using a hybrid counting sort." Journal of Computational Physics 173, no. 2 (2001): 393-411.

[27] H. Chen, S. Madaminov, M. Ferdman and P. Milder, "Sorting Large Data Sets with FPGA-Accelerated Samplesort," 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), San Diego, CA, USA, 2019, pp. 326-326.

[28] Gebala M., Kik M. (2008) Counting-Sort and Routing in a Single Hop Radio Network. In: Kutylowski M., Cichon J., Kubiak P. (eds) Algorithmic Aspects of Wireless Sensor Networks. ALGOSENSORS 2007. Lecture Notes in Computer Science, vol 4837. Springer, Berlin, Heidelberg.

Authorized licensed use limited to: University College Dublin. Downloaded on March 21,2025 at 12:21:26 UTC from IEEE Xplore. Restrictions apply.